Lesson 11 Teacher's Key: Wave After Wave

Scratch - Instructor Reference Only

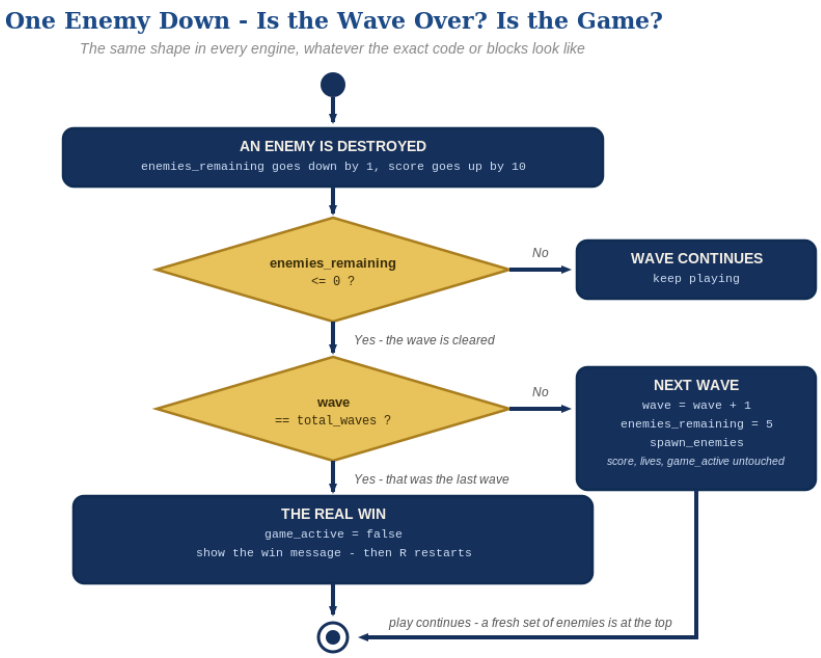
VIDEO GAME CREATION

What This Lesson Teaches

Splitting a single-condition check (Lesson 6's 'are all enemies gone?') into a two-condition decision ('gone, AND was that the last wave?') - the first time this course asks students to nest a decision inside another decision - and reusing one 'make the enemies' routine from a new caller.

Level 1 - Full Worked Solution

- The spawn script already exists from Lesson 10. This week only its hat block changes, from [start game] to [spawn enemies], which turns it into a reusable routine with two callers: the end of GameManager's start-game script, and the else branch of the win check.
- GameManager (Rule 13) owns wave and total_waves and makes every decision. The Enemy sprite still only reports and spawns what it is asked to.
- The delete-clones script stays on [start game]. Clones hear broadcasts, so on a restart every old clone deletes itself before GameManager's [spawn enemies] arrives.



Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

Why This Structure

- Reusing the broadcast mechanism keeps this lesson consistent with everything Scratch has done since Lesson 6 - no new Scratch concept, just a new broadcast and a nested if.
- Naming the broadcast spawn enemies matches spawn_enemies in Godot and GameMaker and SpawnEnemies in Unity (Rule 12), so students in different engines can compare.
- Why the last-hit clone does not double-spawn: it broadcasts [enemy hit] and deletes itself in the same moment, so by the time GameManager broadcasts [spawn enemies] it is gone. Only the original Enemy sprite hears it as a spawner.

Level 2 / Level 3 Talking Points

- Ask students to trace wave 2's clear out loud: enemies_remaining hits 0, wave (1) is compared with total_waves (3), they differ, so wave becomes 2, enemies_remaining resets to 5 and [spawn enemies] is broadcast again.
- For strong students: what would make each wave harder? (More enemies, faster movement.) That is explicitly Lesson 14's job, but the wave counter built today is exactly what difficulty scaling will key off.
- The Stalemate (Level 3): do NOT hand over the fix. It was Lesson 10's Level 3 and is deliberately left in so it becomes irritating enough that students go back and fix it. Waves make it worse: after any escape, enemies_remaining can never reach 0, so that wave never ends - at any of the three waves. Confirm it is real if a student reports it, name it

(the Stalemate), and nudge: 'what has to be true for a wave to be over?' The worked fix below makes an escape also resolve an enemy, and routes both the kill path and the escape path through ONE wave-cleared decision. Accept any design that guarantees every wave ends in exactly one outcome. Test: 3 kills and 2 escapes, 4 and 1, 0 and 5 (the last should be a loss only if lives run out), in each of the three waves.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no blocks change.
// Checks: (1) clear all five three times: waves 1 and 2 continue, wave 3
wins;
// (2) lose the last life in wave 2: Game Over, no new wave;
// (3) R after a win and after a loss: wave 1, score 0, lives 3, exactly
five enemies.
```

Level 3 - complete code (additions on top of Level 1 highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code. Methods shown are the final versions of everything Level 3 touches.

```
// GameManager - one home for the wave-cleared decision (Stalemate fixed):
when I receive [enemy hit]
change enemies_remaining by -1
change score by 10
if enemies_remaining = 0 then
    broadcast [wave cleared]

when I receive [enemy escaped]
change lives by -1
change enemies_remaining by -1
if lives = 0 then
    set game_active to false
    broadcast [game over]
else
    if enemies_remaining = 0 then
        broadcast [wave cleared]
    else
        start sound [sfx_lifelost]  // your Lesson 8 Level 3 sound

when I receive [wave cleared]
if wave = total_waves then
    set game_active to false
    broadcast [you win]
else
    change wave by 1
    set enemies_remaining to 5
    broadcast [spawn enemies]

// Player sprite - wave announcement:
when I receive [spawn enemies]
if wave > 1 then
    say (join [Wave ] (wave)) for 1 seconds
```

Common Student Mistakes

- Game-state drift: wave not reset to 1 in the start-game script, so a restart after wave 3 begins at wave 3. Walk through the Handout's state check.

- Checking only enemies_remaining = 0 without the wave comparison, so clearing wave 1 immediately wins.
- Broadcasting [spawn enemies] outside the else branch, spawning an extra wave on the winning clear.
- Leaving a spawn script on [start game] as well as [spawn enemies]: ten enemies per wave.
- A student reports a wave that never ends: that is the Stalemate. Do not hand over the fix.